

FULL MARKING SCHEME

IS2200

IS Analysis and Design for Business

Winter 2024 Examination · Paper 1

Module	IS2200 — IS Analysis and Design for Business
Session	Winter 2024
Duration	1.5 hours
Structure	Section A (compulsory) · one question from Section B
Total marks	100 (50 + 50)

SECTION A — COMPULSORY

Question 1

(a) A use case diagram outlining the identified users, and how they interact with the system. Clearly **(10 marks)** label the users, use cases, and associations.

FULL MODEL ANSWER

Scope note. Question 1 is one extended scenario (Amazon's order-fulfilment system) developed across parts (a)–(c). The entities and processes identified in (a) and (b) are carried through and decomposed further in (c), rather than re-derived from scratch, matching how an examiner reading the scenario once would expect the three diagrams to relate to one another.

Reading the scenario identifies three external actors, each triggering a distinct cluster of system functionality:

- **Customer** — browses the website/app, adds items to the cart, places the order (inputting payment and shipping preferences), and tracks the shipment.
- **Warehouse Staff** — locate items from inventory, verify order details, and pack the order for shipment, guided by internal systems.
- **Shipping Provider** — an external third-party delivery service engaged to collect and deliver the package, and to update delivery status in real time.

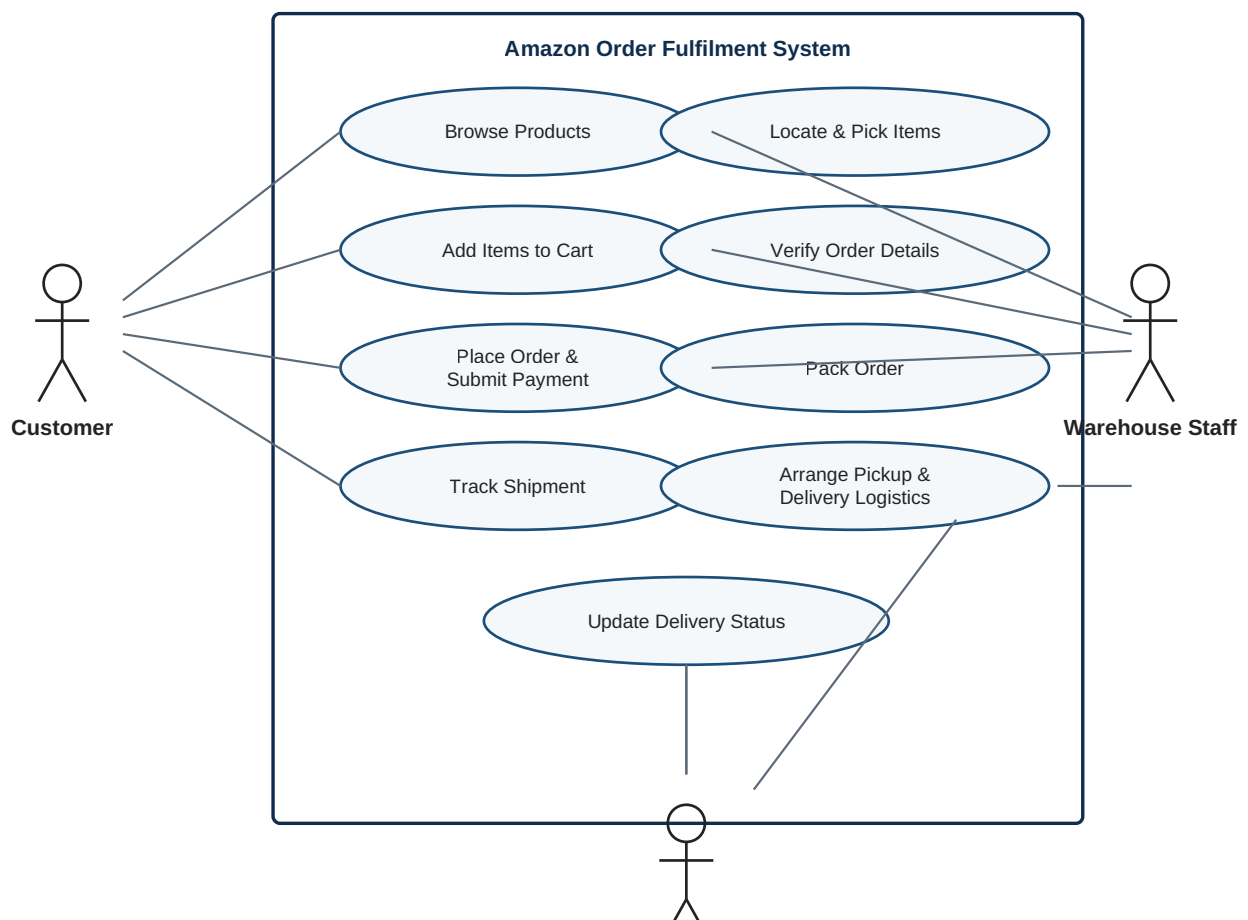


Figure 1(a). Use case diagram for the Amazon order-fulfilment system, showing three actors and nine associated use cases.

"Place Order & Submit Payment" is modelled as a single use case, since the scenario describes payment submission as one step within placing an order rather than a separate goal the customer initiates independently. Each association is an unlabelled participation line, per standard UML convention — only *who takes part in which use case* is shown here, not the data exchanged (that is the job of the DFDs in parts (b) and (c)).

Mark Allocation — (a)

Component	Marks	Criteria for full marks
All three actors correctly identified	3	Customer, Warehouse Staff, and Shipping Provider all present and correctly labelled as actors.
Use cases correctly identified and meaningfully named	4	Use cases reflect distinct goals from the scenario, named as verb phrases; no major function missing or duplicated.
Associations correctly drawn	2	Every use case connected to at least one actor; no actor left unconnected; no association to an unrelated use case.
Correct UML notation	1	System boundary box, ellipse use cases, and stick-figure actors used correctly and consistently.
Total	10	—

(b) Create a context level data flow diagram to provide an overview of the system's interactions with **(15 marks)** external entities. Label all elements appropriately.

FULL MODEL ANSWER

The context-level DFD shows Amazon's order-fulfilment system as a single process, bounded by the three external entities identified in part (a) and the major data flows crossing the system boundary to and from each of them.

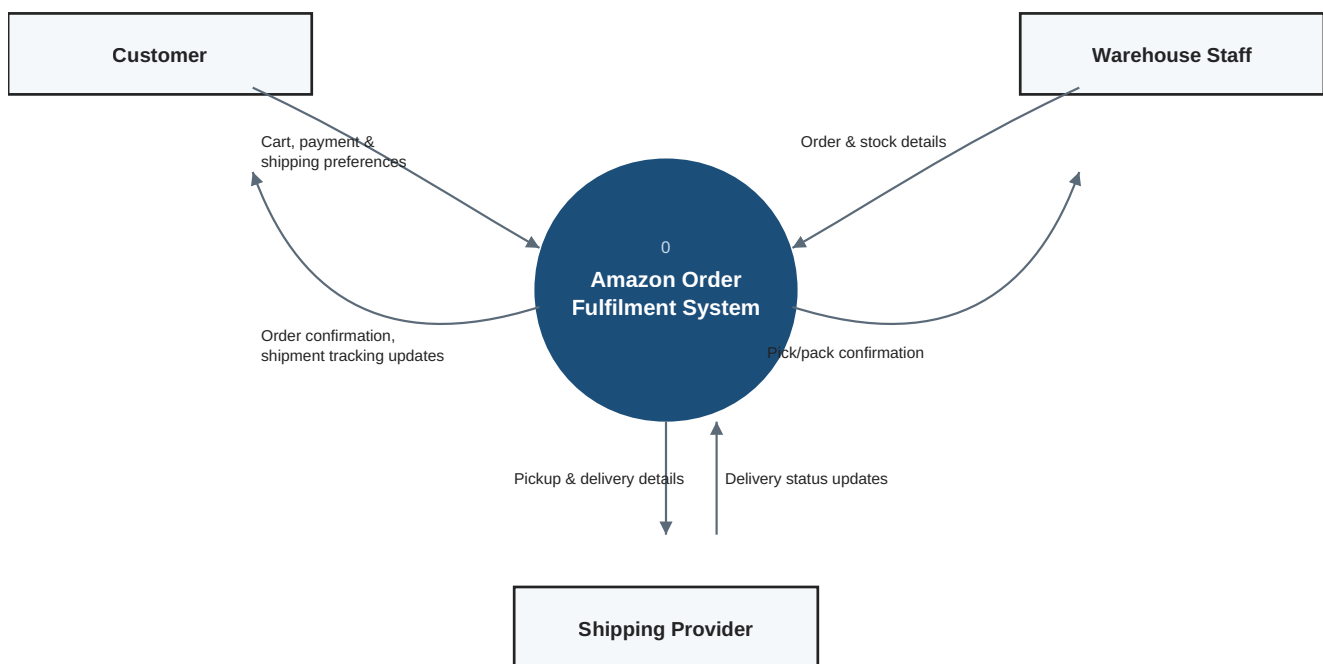


Figure 1(b). Context-level DFD: the Amazon order-fulfilment system as a single process (0), bounded by three external entities.

Data flows are labelled and directional: *browsing, cart and payment/shipping preference data* in from the Customer, and *order confirmation plus real-time shipment tracking updates* back out; *order and stock details* out to Warehouse Staff, and *pick/pack confirmation* back; *pickup and delivery logistics details* out to the Shipping Provider, and *delivery status updates* back — matching the scenario's own statement that this real-time exchange "plays a key role in ensuring accurate and timely delivery."

Mark Allocation — (b)

Component	Marks	Criteria for full marks
Single process correctly represents the whole system	3	Exactly one process, numbered 0, labelled with the system name; no internal detail shown.
All three external entities correctly identified	6	2 marks each: Customer, Warehouse Staff, and Shipping Provider all present and correctly drawn as external entities.
Data flows correctly labelled and directional	4	At least one inbound and one outbound flow per entity, sensibly named from the scenario's own wording.
Correct DFD notation used throughout	2	Circle/rounded shape for the process, squares/rectangles for entities, labelled directional arrows; no data-store symbol at this level.
Total	15	—

(c) Expand on the context level data flow diagram by creating a level 0 data flow diagram.

(25 marks)

FULL MODEL ANSWER

Opening up process 0 into its major internal processes, while keeping every external entity and flow from part (b) unchanged (the balancing rule), gives four processes that follow the scenario's own three paragraphs:

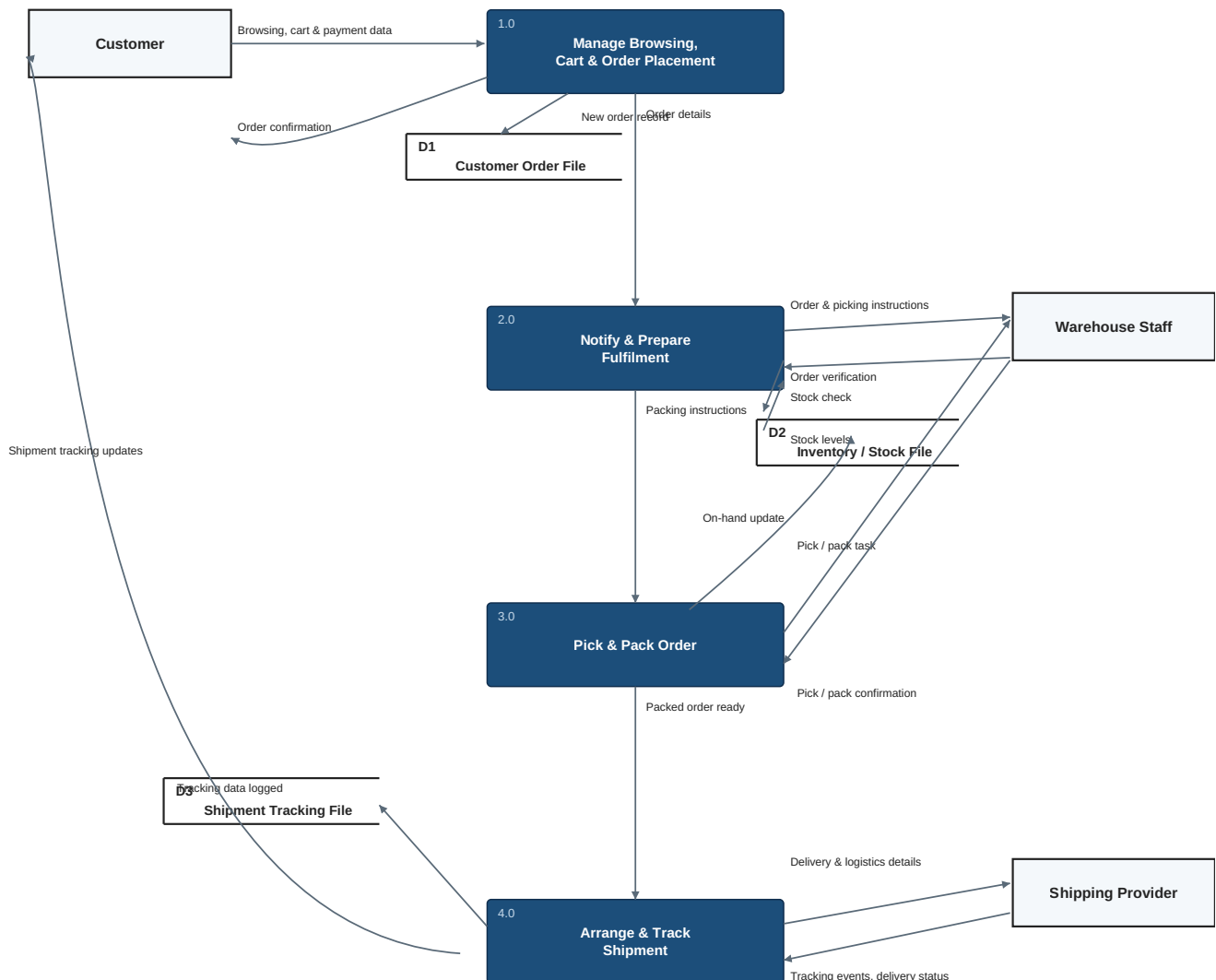


Figure 1(c). Level 0 DFD: decomposition of the Amazon order-fulfilment system into four numbered processes and three data stores.

Process	Covers (from the scenario)
1.0 Manage Browsing, Cart & Order Placement	Customer browses, adds items to cart, inputs payment/shipping information, confirms the order.
2.0 Notify & Prepare Fulfilment	System notifies the relevant warehouse; internal systems guide staff on which items to pick and verify order details.
3.0 Pick & Pack Order	Warehouse staff locate items from inventory, pack them, and prepare them for shipment.
4.0 Arrange & Track Shipment	Shipping provider engaged for pickup/logistics; real-time delivery status exchanged and surfaced to the customer.

Three data stores hold the persistent data the scenario names explicitly: **D1 Customer Order File** (the order record, items, payment/shipping choices), **D2 Inventory/Stock File** (stock levels and order specifics the scenario flags as "crucial for keeping operations smooth"), and **D3 Shipment Tracking File** (the real-time tracking data exchanged between Amazon and the shipping provider).

As with the equivalent question on the Winter 2025 paper, this module's own lecture slides use "Context Level (Level 0)" as one combined term for the single-process diagram, and label the next decomposition step "Level 1." This answer follows the exam's own wording literally, so part (c) here is the diagram the lecture slides would call "Level 1."

Mark Allocation — (c)

Component	Marks	Criteria for full marks
Processes correctly identified, numbered, and decomposed from the context diagram	10	4 major processes present (2.5 marks each), each a genuine transformation of data, consistently numbered 1.0–4.0.
Data stores correctly identified and labelled	6	D1–D3 all present, each holding data genuinely described as persisted in the scenario, correctly drawn with open-ended store notation.
Data flows accurately connect processes, stores and entities	6	Flows correctly labelled and directed; no data store connected directly to another store or external entity without an intervening process.
Consistency (balancing) with the context diagram in part (b)	2	Same three external entities appear; no external flow from part (b) is dropped or contradicted.
Correct DFD notation throughout	1	Rectangles for processes, open rectangles for stores, squares for entities, labelled directional arrows.
Total	25	—

Question 1 Total

50 marks

SECTION B · OPTION A

SECTION B — ANSWER ONE QUESTION FROM THIS SECTION

Question 2

- (a)** Discuss these six key decisions made during the DIA baggage system project that led to its failure. Use specific examples from the case study to support your arguments. **(20 marks)**

FULL MODEL ANSWER

Sourcing note. The paper refers to "six key decisions... explored in class," but the uploaded course pack does not include a dedicated Denver International Airport (DIA) case-study lecture, so the specific six decisions and phrasing used in your lecture cannot be verified against it. The six decisions below are the ones most consistently documented across the standard published case material on this project (the Harvard Business School case series by Applegate, Montealegre, Knoop & Nelson; Montealegre & Keil's 2000 academic analysis; and the Callear Consulting case review) — they are very likely to overlap heavily with what was covered in class, but you should cross-check the specific phrasing and any class-specific framing against your own lecture notes before treating this as the final word on "the six.")

#	Decision	Why it contributed to failure
1	Committing to one fully-integrated, airport-wide system instead of a smaller or phased scope	The system was designed to automate baggage handling across all three concourses for every airline, making it, in its own case material's words, the most complex automated baggage system ever attempted — roughly ten times larger than any system built before it — rather than starting with a single concourse or airline.
2	Awarding the contract to BAE Automated Systems very late relative to the airport's fixed opening date	The baggage-system contract was only finalised in 1992, leaving BAE roughly two years to design and build a system of unprecedented complexity against an October 1993 opening date that had already been fixed and publicly announced.
3	Repeatedly changing scope and design during construction without re-baselining schedule or cost	United Airlines, for example, requested design changes such as removing an entire loop of track from one concourse partway through the build; each change absorbed schedule float without the overall deadline being revisited.
4	Ignoring outside expert warnings that the timeline and design were not feasible	Independent consultants advised early on that there was not enough time to build a system of this complexity; those warnings were not acted on, and the underlying technical problem (dynamically routing empty carts to over 100 pickup points, known as "line balancing") was left effectively unsolved at scale.
5	Holding to a fixed, politically-announced opening date rather than a technically-justified one	Denver's mayor had publicly committed to an opening date, which created strong incentives to keep reporting the project as on-track and discouraged the kind of early escalation that might have triggered a scope or schedule change sooner.
6	Treating a full working automated system as the only acceptable outcome , with no tested fallback	A conventional manual tug-and-cart backup system was only commissioned very late, at a cost of roughly \$75–86 million, once it was already clear the automated system could not be ready — rather than being planned in from the start as a contingency.

Taken together, these six decisions describe a project that escalated its own commitment to a single, high-risk path (a fully automated, airport-wide system, on a fixed political deadline) at every point where a course correction was still

possible — which is precisely the "de-escalation of commitment" framing used in the academic case literature on this project.

Mark Allocation — (a)

Component	Marks	Criteria for full marks
Six decisions correctly identified and explained with a specific example	18	3 marks per decision: the decision is clearly stated, explained in terms of why it increased failure risk, and grounded in a specific fact/example from the case (not a generic project-management truism).
Overall coherence and synthesis	2	Answer connects the six decisions into a coherent account of how the project's problems compounded, rather than presenting six disconnected bullet points.
Total	20	—

(b) Provide a comparison of processes involved in the software development lifecycle model, scrum, **(30 marks)** and Kanban as explained in class.

FULL MODEL ANSWER

Waterfall SDLC, Scrum, and Kanban all sequence the same underlying software work — understanding requirements, building, and verifying a system — but they structure that sequence, assign responsibility for it, and respond to change in fundamentally different ways.

Dimension	Waterfall SDLC	Scrum	Kanban
Process structure	Strictly sequential, phase-gated: requirements → design → implementation → testing → deployment → maintenance. Each phase is signed off before the next begins.	Iterative and time-boxed: work is planned, built, and reviewed in fixed-length sprints (commonly 1–4 weeks), each producing a potentially shippable increment.	Continuous flow, not time-boxed: work items move one-by-one through a visual board of workflow stages as capacity allows, with no fixed iteration boundary.
Roles	Typically a project manager/analyst-led hierarchy with distinct phase specialists (analysts, designers, developers, testers) handing off work between phases.	Three defined roles: Product Owner (owns and prioritises the backlog), Scrum Master (facilitates process and removes blockers), and a self-organising Development Team.	No prescribed roles; existing team structure and responsibilities are generally left as they are — Kanban changes how work is visualised and pulled, not who does what.
Planning & cadence	One large, upfront planning effort; the full scope and schedule are meant to be fixed before implementation starts.	Planning happens every sprint (sprint planning), with a daily stand-up, a sprint review, and a sprint retrospective forming a fixed ceremonial cadence.	No fixed planning cadence; new work is pulled into the board continuously as capacity (a Work-in-Progress limit) frees up.
Handling change	Change is expensive and discouraged once a phase is signed off; a change typically means re-opening an earlier phase's documentation.	Change is accommodated at sprint boundaries; the backlog can be re-prioritised between sprints, but scope inside an active sprint is meant to stay stable.	Change is accommodated continuously; a new or re-prioritised item can be pulled in as soon as WIP capacity allows, without waiting for an iteration boundary.
Delivery cadence	One delivery, at or near the end of the project, after all phases are complete.	A working increment at the end of every sprint, giving regular, predictable delivery checkpoints.	Continuous delivery of individual items as each one completes, rather than in batches.
Best suited for	Well-understood, stable requirements where the cost of late change is high (e.g. safety-critical or heavily regulated systems).	Product development where requirements will evolve and regular stakeholder feedback on working increments is valuable.	Ongoing operational or support work (e.g. a maintenance or ticket queue) where work arrives unpredictably and throughput/flow matters more than fixed releases.

The practical implication of these differences is that Waterfall optimises for predictability and documentation at the cost of flexibility, Scrum trades some predictability for regular feedback and the ability to re-prioritise between sprints, and Kanban trades fixed cadence entirely in exchange for maximum responsiveness to continuously arriving work — which is why the "right" choice depends on how stable the requirements are and how the work actually arrives, rather than one approach being universally superior to the others.

Waterfall SDLC and Agile are covered directly in this module's lecture material; Kanban specifically is not covered there in the extracted course pack. The Kanban content above reflects standard, widely-taught software-engineering practice rather than a specific slide in your course material — check it against anything your lecturer said about Kanban specifically in class.

Mark Allocation — (b)

Component	Marks	Criteria for full marks
Process structure compared accurately across all three	5	Correctly distinguishes phase-gated (Waterfall), time-boxed iterative (Scrum), and continuous-flow (Kanban) structures.
Roles compared accurately across all three	5	Correctly identifies Scrum's three named roles versus Waterfall's phase-based hierarchy and Kanban's role-agnostic approach.
Planning and cadence compared accurately across all three	5	Correctly distinguishes one-off upfront planning, sprint-based ceremonial cadence, and continuous pull-based planning.
Handling of change compared accurately across all three	5	Correctly explains why change is costly in Waterfall, boundary-gated in Scrum, and continuous in Kanban.
Delivery cadence compared accurately across all three	5	Correctly distinguishes single end-of-project delivery, per-sprint increments, and continuous per-item delivery.
Best-fit context correctly identified for all three	5	Gives a defensible real-world scenario type for each approach, not just a restatement of its mechanics.
Total	30	—

Question 2 Total

50 marks

SECTION B · OPTION B

SECTION B — ANSWER ONE QUESTION FROM THIS SECTION

Question 3

(a) Draw a network diagram for the activities.

(30 marks)

FULL MODEL ANSWER

Scope note. Parts (a)–(c) all build on the single project schedule in Table 1, and part (c)'s testing-phase risk data is separate, updated information given after part (b) — it is not derived from Table 1's activity list. Each part is answered independently below using the correct source data for that part.

Using the module's own network-diagram convention, each activity is drawn as a labelled node, with a forward pass computing each activity's earliest start/finish (T_E) by summing durations along every predecessor path, and a backward pass (computed in part (b)) computing each activity's latest allowable start/finish (T_L) working back from the project's overall duration.

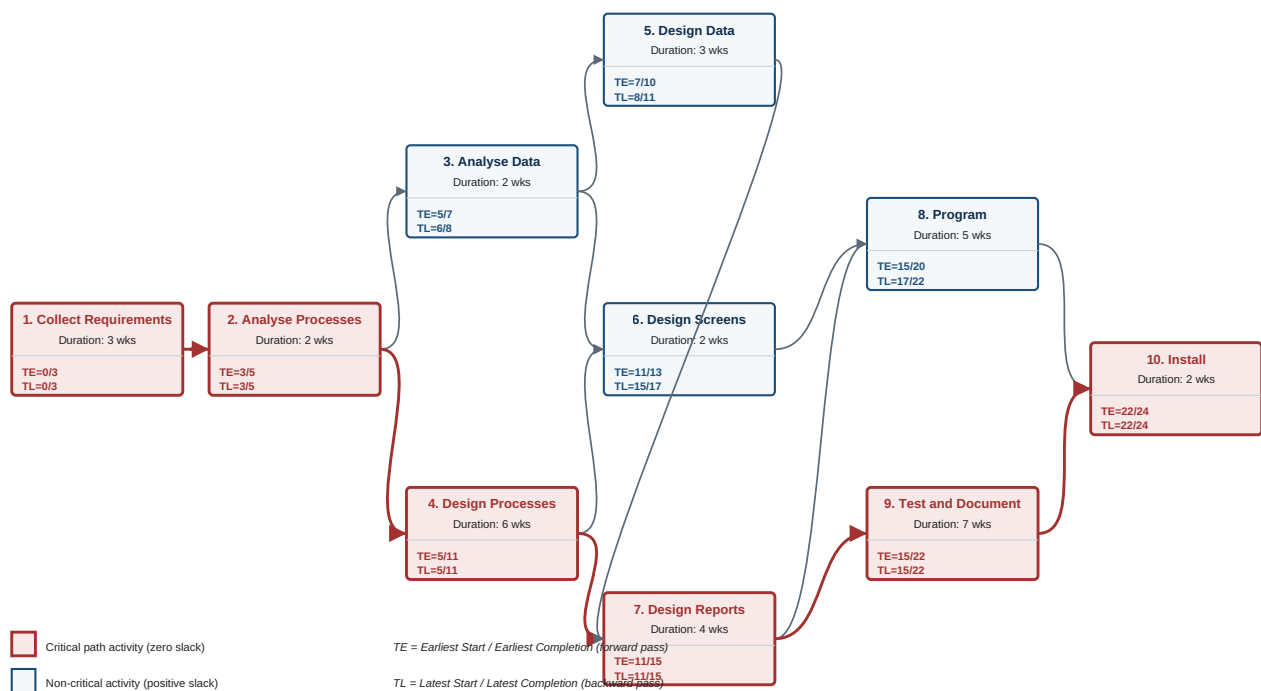


Figure 3(a)/(b). Activity-on-node network diagram for the HR system project, with forward-pass (T_E) and backward-pass (T_L) values shown on every node, and the critical path highlighted in red.

Activity	Description	Duration (wks)	Predecessors	T_E (ES/EF)
1	Collect Requirements	3	—	0 / 3
2	Analyse Processes	2	1	3 / 5
3	Analyse Data	2	2	5 / 7
4	Design Processes	6	2	5 / 11
5	Design Data	3	3	7 / 10
6	Design Screens	2	3, 4	11 / 13
7	Design Reports	4	4, 5	11 / 15
8	Program	5	6, 7	15 / 20
9	Test and Document	7	7	15 / 22
10	Install	2	8, 9	22 / 24

Where an activity has more than one predecessor (e.g. Activity 6 depends on both 3 and 4), its earliest start is the *later* of its predecessors' earliest finish times — Activity 6 cannot start until both Activity 3 (finishing at week 7) and Activity 4 (finishing at week 11) are done, so its earliest start is week 11, not week 7. The forward pass shows the project's earliest overall completion is **24 weeks** (Activity 10's T_E finish).

Mark Allocation — (a)

Component	Marks	Criteria for full marks
All 10 activities correctly represented with durations	8	Every activity from Table 1 present as a node, correctly labelled with its number, name, and duration.
Precedence relationships correctly drawn	10	Every arrow matches Table 1's predecessor column exactly; no missing or extra dependency; correct handling of multi-predecessor activities (6, 7, 10).
Forward pass (T_E / earliest start & finish) correctly calculated	6	Every node's earliest start/finish correctly computed, taking the maximum of all predecessor finish times where more than one predecessor exists.
Diagram clarity and correct AON notation	6	Node-and-arrow (activity-on-node) format used consistently, matching the module's own network-diagram convention; legible layout with no ambiguous overlaps.
Total	30	—

(b) Show the critical path.**(10 marks)****FULL MODEL ANSWER**

The backward pass starts from the project's overall duration (24 weeks, from part (a)) and works right-to-left, computing each activity's latest allowable finish as the *earliest* of its successors' latest start times. Slack (or float) for each activity is then latest finish minus earliest finish (equivalently, latest start minus earliest start); any activity with zero slack lies on the critical path, because it has no room to slip without delaying the whole project.

Activity	T_E finish	T_L finish	Slack	On critical path?
1. Collect Requirements	3	3	0	Yes
2. Analyse Processes	5	5	0	Yes
3. Analyse Data	7	8	1	No
4. Design Processes	11	11	0	Yes
5. Design Data	10	11	1	No
6. Design Screens	13	17	4	No
7. Design Reports	15	15	0	Yes
8. Program	20	22	2	No
9. Test and Document	22	22	0	Yes
10. Install	24	24	0	Yes

The critical path is therefore **1 → 2 → 4 → 7 → 9 → 10** ($3 + 2 + 6 + 4 + 7 + 2 = \mathbf{24 \text{ weeks}}$), shown highlighted in red on Figure 3(a)/(b) above. Activities 3, 5, 6, and 8 all carry positive slack (1, 1, 4, and 2 weeks respectively) and can therefore be delayed by up to that amount without pushing back the project's 24-week completion date — useful information for the project manager if resources need to be temporarily reallocated away from those activities.

Mark Allocation — (b)

Component	Marks	Criteria for full marks
Critical path correctly identified	6	States the path 1-2-4-7-9-10 (or equivalent zero-slack justification for each activity on it).
Total project duration correctly stated	2	States 24 weeks, consistent with the forward pass in part (a).
Correct justification via slack/float	2	Explains that critical activities have zero slack ($T_e = T_s$), distinguishing them from the non-critical activities with positive slack.
Total	10	—

(c) Calculate the expected value of the testing phase.**(10 marks)****FULL MODEL ANSWER**

Expected value is the sum, over every possible outcome, of that outcome's probability multiplied by its financial value:

Outcome	Probability	Value (€)	Probability × Value
Successful Testing	25%	+15,000	+3,750
Minor Bugs Found	30%	−3,000	−900
Moderate Bugs Found	25%	−7,000	−1,750
Major Bugs Found	10%	−20,000	−2,000
Critical Bugs Found	10%	−40,000	−4,000
Expected Value	100%	—	−€4,900

The five probabilities sum to 100%, confirming the outcomes are exhaustive and mutually exclusive. The expected value of the testing phase is **−€4,900**: averaged across many projects facing this same risk profile, the testing phase should be expected to cost the project €4,900 net, even though the single most likely individual outcome is Minor Bugs Found (30%), not the worst case. As in any EV analysis, this figure is a planning benchmark for budgeting contingency, not a prediction of what will happen on this specific project.

Mark Allocation — (c)

Component	Marks	Criteria for full marks
Correct formula/method applied	2	States or clearly applies $EV = \sum(\text{probability} \times \text{value})$.
Correct substitution of all five outcomes	4	Each of the 5 probability×value products correctly calculated (0.8 marks each).
Correct final arithmetic and expected value stated	3	Products correctly summed to −€4,900, with correct sign.
Clear presentation and brief interpretation	1	Working shown in a table or equivalent structured form; sign/meaning of a negative EV briefly interpreted.
Total	10	—

Question 3 Total**50 marks**